DATA STRUCTURES AND ITS APPLICATIONS

Graphs

Saritha

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Connectivity of the graph

Saritha

Department of Computer Science & Engineering

Applications of BFS

Application of BFS

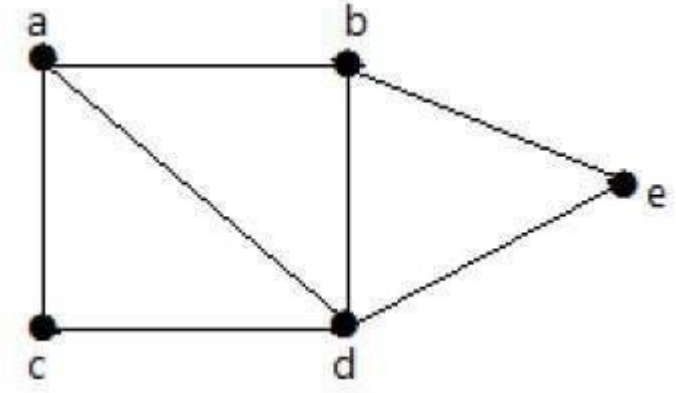
- Finding the shortest path
- Social Networking websites like twitter, Facebook etc.
- GPS Navigation system
- Web crawlers
- Finding a path in network
- In Networking to broadcast the packets

Connectivity of graph

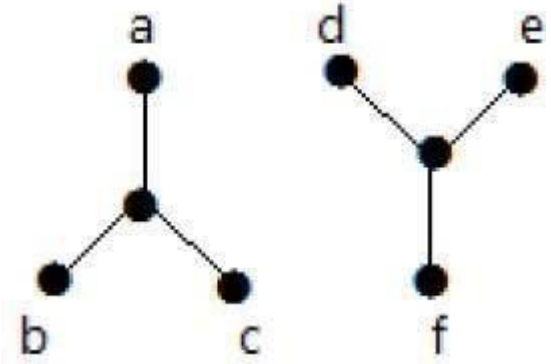
Connectivity of graph

- Connectivity refers to connection between two or more nodes or things

Connected graph



Disconnected graph

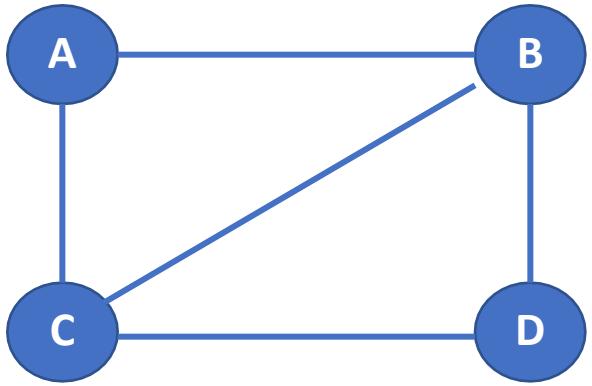


Connectivity of the graph

- A graph is connected if there is a path between every pair of vertices.
- The graph that is not connected is called disconnected graph.
- In connected graph there is no unreachable vertex.
- In the area of Information Technology the connectivity refers to internet connectivity through which various devices are connected to global network

Connectedness in the Direct graph using adjacency matrix

- To check whether a graph is connected or not, we traverse the graph using either bfs traversal or dfs traversal method
- After the traversal if there is at-least one node which is not marked as visited then that graph is disconnected graph
- For example: Below graph is connected



A

A B			
B	1	0	1	1
C	1	1	0	1
D	0	1	1	0

Connectedness in the Direct graph using adjacency matrix

Procedure to check the whether a graph is connected or not using adjacency matrix

- Read the adjacency matrix .
- Create a visited [] array. Start DFS/BFS traversal method from any arbitrary vertex and mark the visited vertices in the visited[] array.
- Once DFS/BFS is completed check the visited [] array. if there is at-least one vertex which is marked as unvisited then the graph is disconnected otherwise it is connected.

Function to read adjacency matrix:

```
void read_ad_matr(int graph[][],int n)
{
  int i,j;
  for(i=0;i<n;i++)
   {
    for(j=0;j<n;j++)
     {
      scanf("%d", &graph[i][j]);//reading the values into two dimensional
      array
     }
   }
}
```

Connectivity of the graph using Adjacency Matrix

Function to traverse the graph using DFS

```
void traverse(int u, int visited[])
{
   visited[u] = 1; //mark v as visited
   for(int v = 0; v<n; v++)
  {
     if(graph[u][v])
     {
       if(!visited[v])
       traverse(v, visited);
     }
  }
}
```

Connectivity of the graph using Adjacency Matrix

Function to traverse the graph using bfs:

```
void traverse(int s, int visited[],int n,int graph[][])
 {
  int f,r,v,q[10]; f=0,r=-
  1;
  q[++r]=s;
  visited[s]=1;// mark the nodes as visited
  while(f<=r)
  {
    s=q[f++];
    for(v=0;v<n;v++)
   {
    if(graph[s][v]==1) //adjacent nodes
     {
```

Connectivity of the graph using Adjacency Matrix


```
    if(visited[v]==0) //nodes not visited
     {
       visited[v]=1; //mark as visited
       q[++r]=v;
     }
}
 }
  }
}
```

Connectivity of the graph using Adjacency Matrix

Function to check whether the graph is connected or not

```
int isConnected()
{
   int vis[NODE]; //for all vertex u as start point, check whether all nodes are visible or not
  for(int u=0; u < NODE; u++)
     {
       for(int i = 0; i<NODE; i++)
            vis[i] = 0; //initialize as no node is visited
       traverse(u, vis); // any traversal method either bfs or dfs
        for(int i = 0; i<NODE; i++)
        {
            if(!vis[i]) //if there is a node, not visited by traversal, graph is not connected
            return 0;
         }
    }
  return 1;}
```

1. A bridge in a graph refers to:

a) A set of vertices with maximum connectivity

b) A set of edges that, if removed, disconnects the graph

c) A cycle formed by vertices

d) A complete subgraph

1. A bridge in a graph refers to:

a) A set of vertices with maximum connectivity

b) A set of edges that, if removed, disconnects the graph

c) A cycle formed by vertices

d) A complete subgraph

2. The edge connectivity of a graph refers to:

a) The number of edges in the graph
b) The maximum number of edges incident to a vertex
c) The minimum number of edges that disconnect the graph
d) The maximum number of edges that form a cycle

2. The edge connectivity of a graph refers to:

a) The number of edges in the graph

b) The maximum number of edges incident to a vertex

c) The minimum number of edges that disconnect the graph

d) The maximum number of edges that form a cycle

3. Between any two vertices, there exists a path. The graph is said to be:

a) Disconnected
b) Strongly connected
c) Biconnected
d) Directed acyclic

3. Between any two vertices, there exists a path. The graph is said to be:

a) Disconnected

b) Strongly connected

c) Biconnected

d) Directed acyclic

4. A strongly connected graph is formally defined as:

a) A graph where each vertex has a path to every other vertex and vice versa
b) A graph with at least one cycle
c) A graph without any isolated nodes
d) A tree structure graph

4. A strongly connected graph is formally defined as:

a) A graph where each vertex has a path to every other vertex and vice versa

b) A graph with at least one cycle

c) A graph without any isolated nodes

d) A tree structure graph

5. If the direction of all edges in a directed graph is reversed, what happens to its strongly connected components?

a) They become disconnected
b) They stay the same
c) They merge into one component
d) They double in number

5. If the direction of all edges in a directed graph is reversed, what happens to its strongly connected components?

a) They become disconnected

b) They stay the same

c) They merge into one component

d) They double in number

THANK YOU

Saritha

Department of Computer Science & Engineering

Saritha.k@pes.edu

9844668963

DATA STRUCTURES AND ITS APPLICATIONS

Graphs

Saritha

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Finding all the paths from a given source to destination

Saritha

Department of Computer Science & Engineering

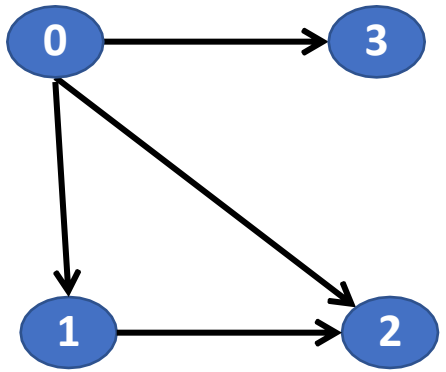
Applications of BFS and DFS

Application of DFS

- Detecting whether a cycle exist in graph.
- Finding a path in a network
- Topological Sorting: Used for job scheduling
- To check whether a graph is strongly connected or not

Path

- Sequence of edges that allows the user to go from vertex A to vertex B



- The paths from vertex 0 to vertex 2:
- 1.0->1->2
- 2.0->2

Finding all the paths from the given source to destination

- **Methodology**
- 1.Start from any traversal Method from a given source node
- 2.Store all the visited vertices in an array
- 3.Once the destination vertex is reached, print all the contents of Array.

Function to print all the paths using DFS traversal method


```
//Function to print all the paths from a given source to
destination void path_find(int source,int destination)
{
    int visited[10]                 //An array to store the vertices as
    visited or not int path[10]          //An array to store the path
    int count=0;
    for(int i=0;i<n;i++)
        visited[i]=0      //initilize all the vertices as not
    visited. printallpaths(source,destination,visited,path);
}
```

Function to print all the paths using DFS traversal method


```
void printallpaths(int u,int d,int visited[10],int path[10])
{
   visited[u]=1;//Mark the current node and and store it in the array
   path path[count]=u;
   count++;
   if(u==d) //if the current vertex is same as the destination then print the array
which has stored the path
    {
      for(i=0;i<count;i++)
            printf("%d",path[i]);
     }
   else // if the current vertex is not the destination
   {
     for(NODE temp=a[u];temp!=NULL;temp=temp->link)
```

Function to print all the paths using DFS traversal method


```
if(!visited[temp->data])
{
   printallpaths(temp->data,d,visited,path);
}
}
count--          //Remove the current vertex from the path[] and mark
it as
unvisited
Visited[u]=0;
}
```

Function to read adjacency list

```
void read_adjacency_list(NODE a[],int n)
{
   int ele,m;
   for(int i=0;i<n;i++)
   {
      printf("enter the number of nodes adjacent to %d:",i);
       scanf("%d",&m);
       if(m==0)
       continue;
       printf("enter the nodes adjacent to %d:",i);
       for(int j=0;j<m;j++)
      {
        scanf("%d",&ele);
       a[i]=insert_rear(ele,a[i]); // insert at rear end
       }   } }
```

Function to insert a node at rear end of the list

Function to insert a node at rear end

```
NODE insert_rear(int v,NODE head)
 {
    NODE temp;
    NODE cur;
    temp=getnode(); // create a node
    temp->info=v;
    temp->link=NULL;
    if(head==NULL)
         return temp;
     cur=head;
   while(cur->link!=NULL)
   {
      cur=cur->link;
    }
   cur->link=temp;
 return(head); }
```

Connectivity of the graph using Adjacency list

Function to create a node

```
NODE getnode()
 {
  NODE temp;
  temp=(NODE)malloc(sizeof(struct node)); //Dynamic
  allocation if(temp==NULL)
  {
     printf("out of
      memory"); return;
   }
  return temp;
 }
```

1. Which graph traversal technique is most suitable for finding all paths from a source to destination?

a) Breadth-First Search (BFS)

b) Depth-First Search (DFS)

1. Which graph traversal technique is most suitable for finding all paths from a source to destination?

a) Breadth-First Search (BFS)

b) Depth-First Search (DFS)

2. In a recursive DFS algorithm to find all paths between two nodes, what is essential to avoid cycles?

a) Using a queue
b) Keeping a visited list or set
c) Sorting adjacency list
d) Using a priority queue

2. In a recursive DFS algorithm to find all paths between two nodes, what is essential to avoid cycles?

a) Using a queue

b) Keeping a visited list or set

c) Sorting adjacency list

d) Using a priority queue

3. Which of the following is a common method to traverse a graph by exploring neighbors first?

a) Depth First Search (DFS)
b) Breadth First Search (BFS)
c) Dijkstra's Algorithm
d) Kruskal's Algorithm

3. Which of the following is a common method to traverse a graph by exploring neighbors first?

a) Depth First Search (DFS)

b) Breadth First Search (BFS)

c) Dijkstra's Algorithm

d) Kruskal's Algorithm

THANK YOU

Saritha
Department of Computer Science & Engineering

Saritha.k@pes.edu

Data Structures and its Applications

V R BADRI PRASAD

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

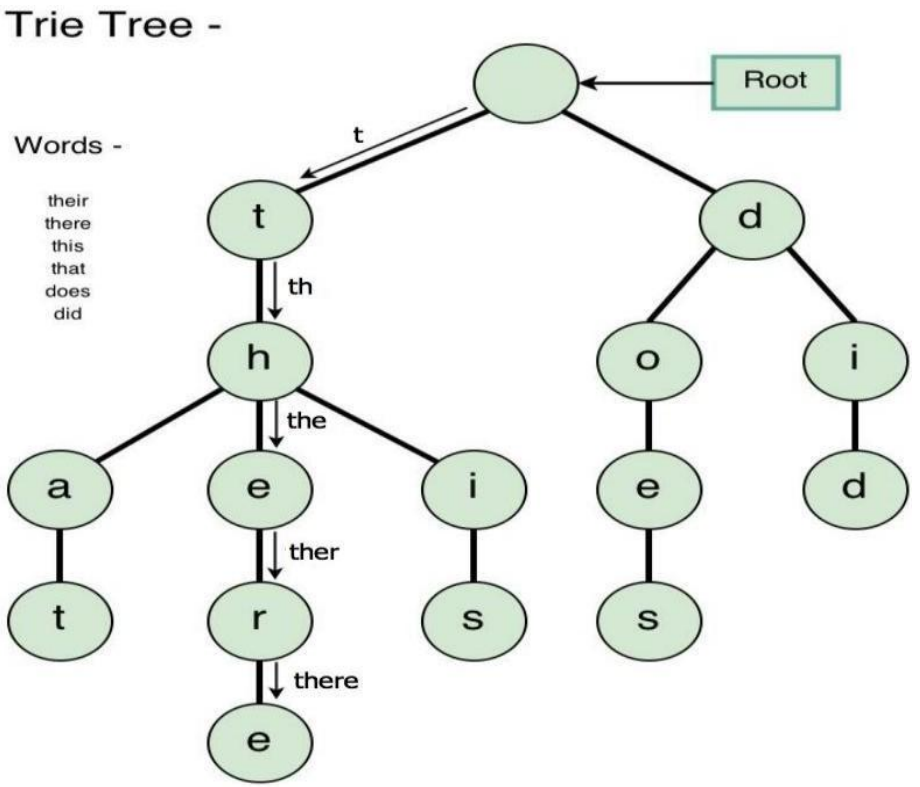
Introduction to TRIE Trees

V R BADRI PRASAD
Department of Computer Science & Engineering

TRIE Trees – An Introduction

- TRIE tree is a digital search tree (also called as Prefix tree), need not be implemented as a binary tree.
- Each node in the tree can contain ‘m’ pointers – corresponding to ’m‘ possible symbols in each position of the key.
- Generally used to store strings.

Examples:



Data Structures and its Applications

TRIE Trees – An Introduction

- A trie, pronounced “try”, is a tree that exploits some structure in the keys

  - e.g. if the keys are strings, a binary search tree would compare the entire strings

    but a trie would look at their individual characters

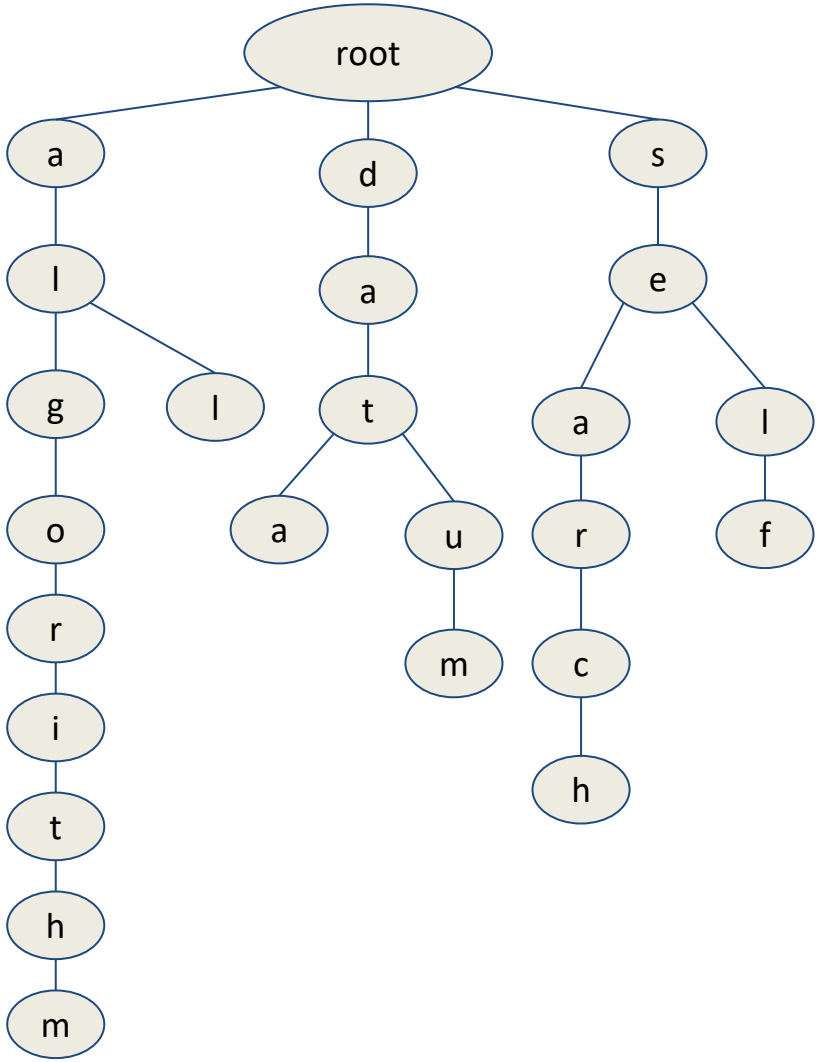
  -A trie is a tree where each node stores a bit indicating whether the string

    spelled out to this point is in the set

- Tries are extremely special and useful data-structure that are based on the *prefix of a string*.
- Strings are stored in a top to bottom manner based on their prefix in a TRIE.
- All prefixes of length 1 are stored at until level 1, all prefixes of length 2 are sorted at until level 2 and so on

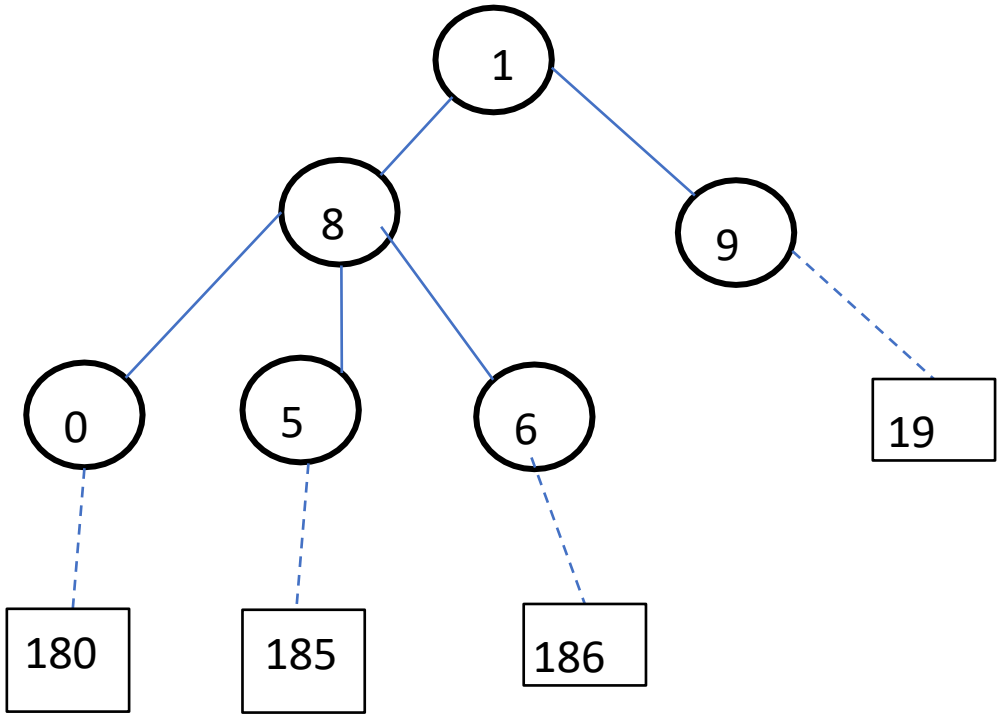
Trie

Construct a Trie for the words: algorithm, data, datum, all, self, sea, search



TRIE Trees – Numeric Keys : Example2

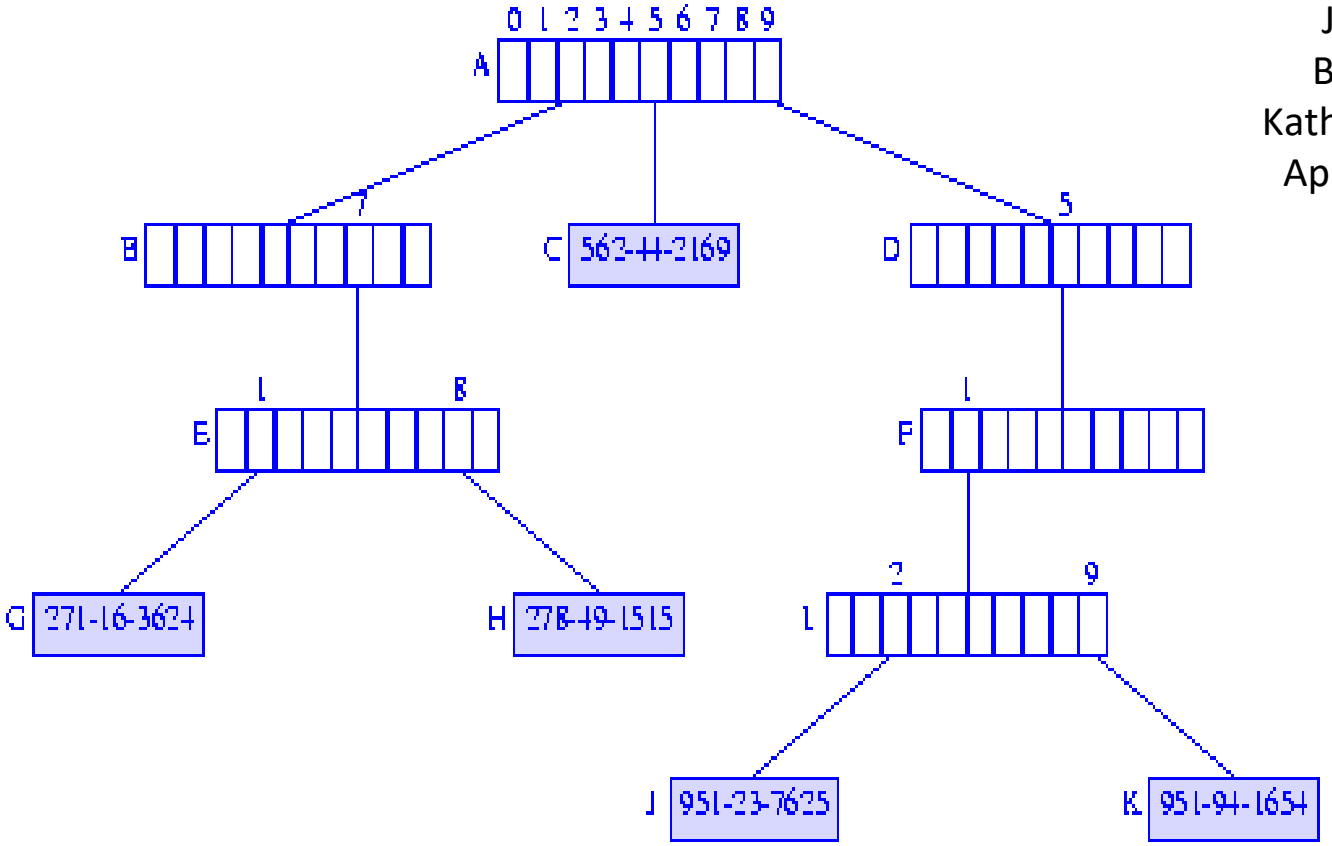
- If the keys are numeric, there would be 10 pointers in a node.



TRIE Trees – Numeric Keys : Example1

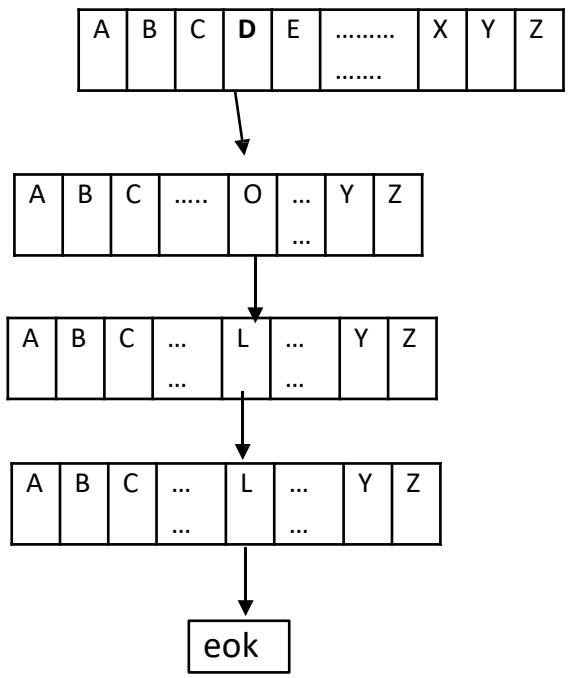
- If the keys are numeric, there would be 10 pointers in a node.
- Consider the SSN number as shown.

Name	**Social Security Number (SS#)**
Jack	951-94-1654
Jill	562-44-2169
Bill	271-16-3624
Kathy	278-49-1515
April	951-23-7625



TRIE Trees – An Introduction

- If the keys are Alphabetic, there would be 26 pointers.



Ex: The word DOLL has been stored as

shown in the figure.

TRIE Trees – An Introduction

- An extra pointer corresponding to eok (end of key) or a flag with each pointer indicating that it point to a record rather than to a tree node. (normally $ symbol is used).
- A pointer in the node is associated with a particular symbol value based on its position in the node.
  - First pointer corresponds to the lowest value.
  - Second pointer to the second lowest and so forth.
- This way of implementation of a digital search tree is called a **TRIE** tree.
- The word ***TRIE*** is extracted from re***trie***val word.

TRIE Trees – Applications, advantages and disadvantages

Applications

- Used in **English dictionaries**
- **Predictive text** input systems
- **Auto-complete** features in cell phones and other gadgets

Advantages

- **Faster** than Binary Search Trees (BST)
- **Easy** to print all stored strings in **alphabetical order**
- Supports **prefix-based searching** (useful for auto-complete)

Disadvantages

- Requires **large memory** for storing all strings
- Involves **many node pointers**, increasing space usage

Suffix Tree

Suffix Trie: A trie containing all the suffixes of the given data

Suffix Tree: A compressed suffix Trie

Suffix Trie – Building

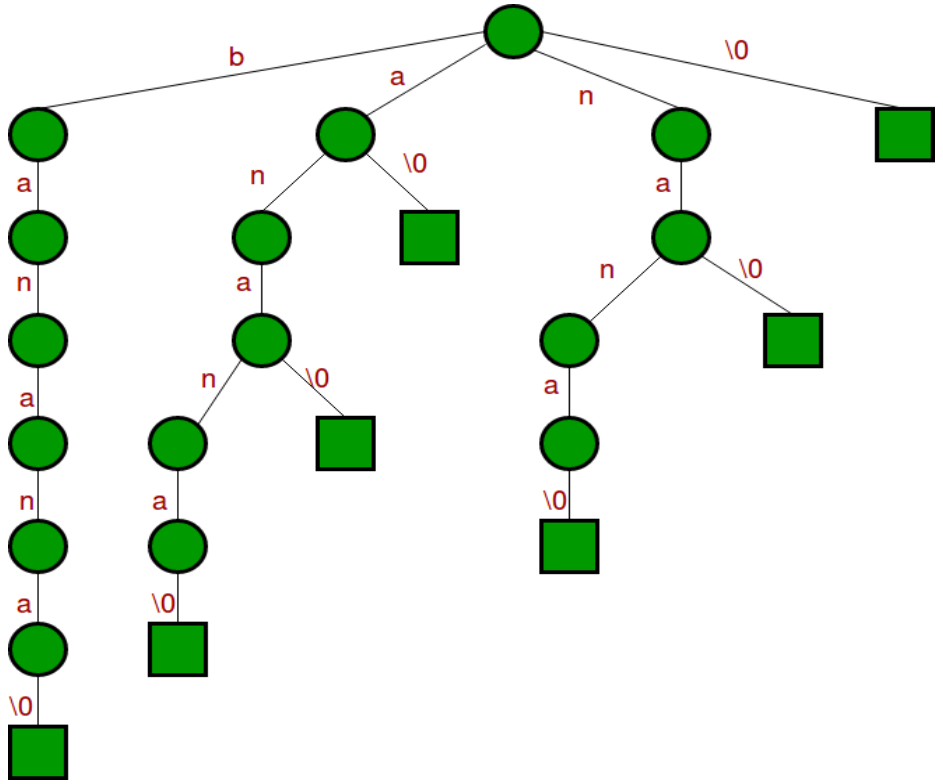
- A Trivial Algorithm for building a suffix tree.
  - Step1 : Generate all suffixes of a given text
  - Step2: Consider all suffixes as individual words and build a compressed trie.

- Example1: "**banana\$**" where '\$' is the string terminating character.

Following are the suffixes of Text "**banana/$**"

"anana/$"
"nana/$"
"ana/$"
"na/$"
"a/$"
"/$"

Suffix trie



Suffix Trie – Building

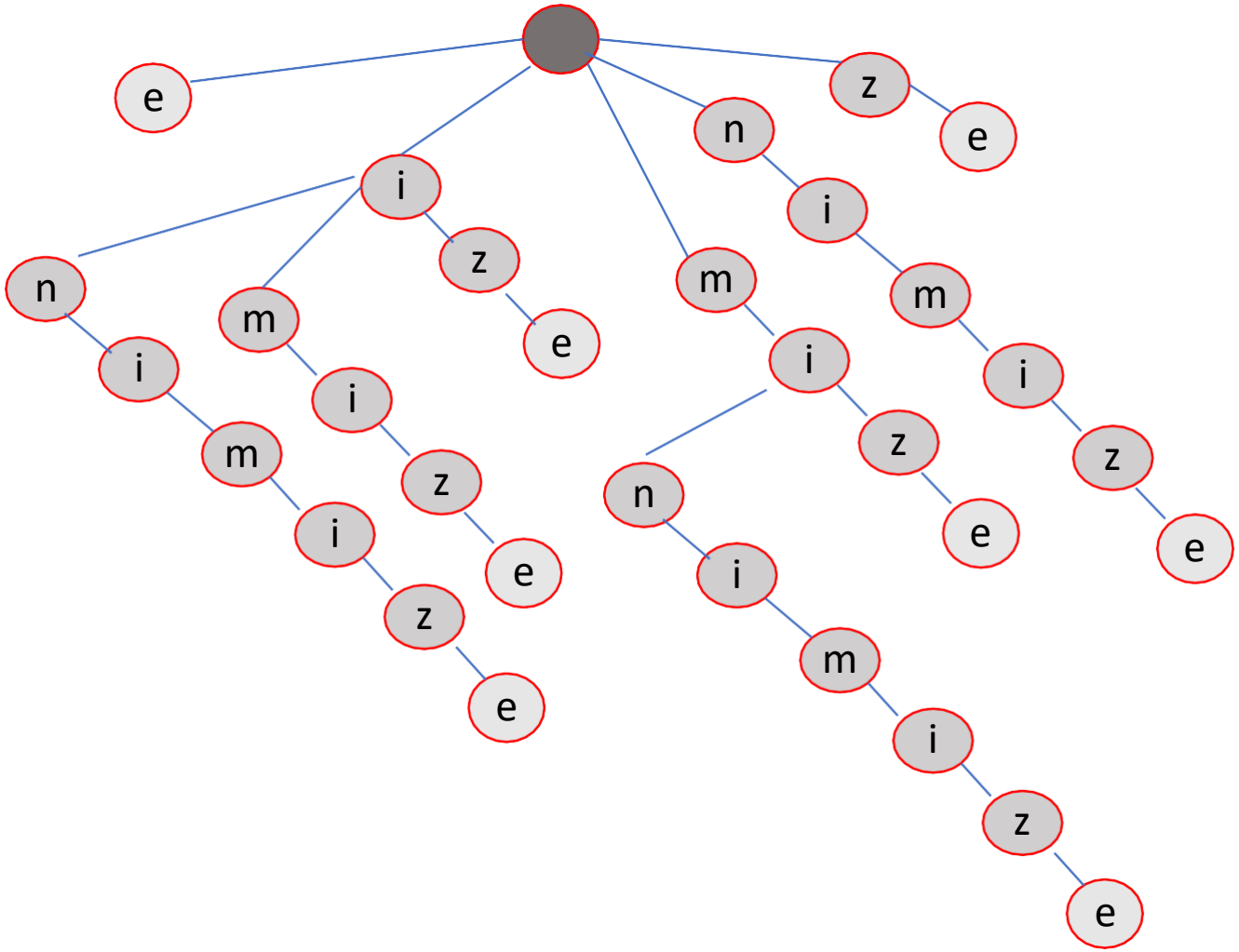
Example 2 : Generate the suffix trie for the word **minimize**

Step1. Generate all the suffixes of the word **minimize**.

e
ze
ize
mize
imiz
e
nimize
inimize
minimiz
e

S - set of strings to include in the suffix trie.

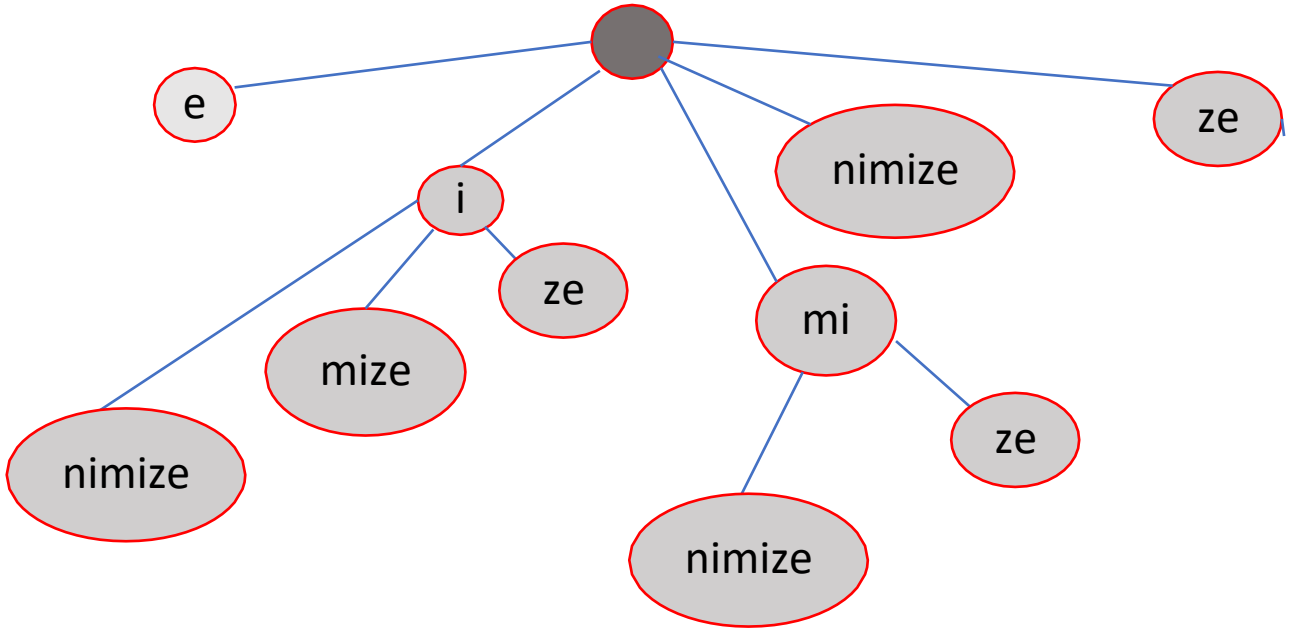
Suffix Trie – Building - for the word minimize



e
ze
ize
mize
imiz
e
nimize
inimize
minimiz
e

S - set of strings to include in the suffix trie

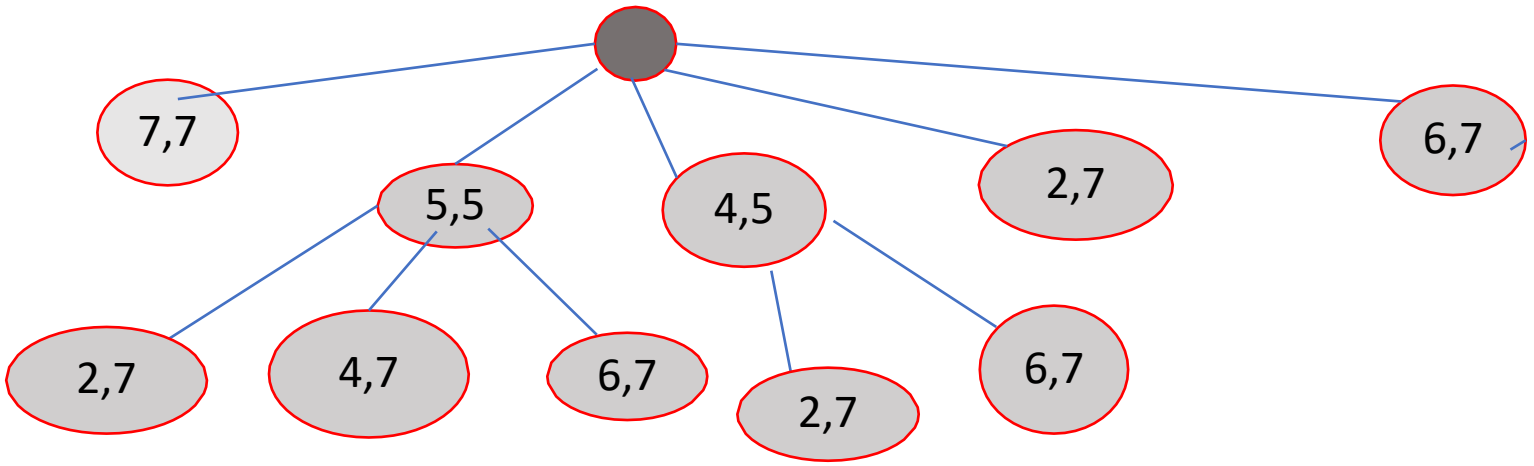
Suffix Trie – Compressed Trie



Suffix Trie – Compressed Trie – using numbers

- Representation of Compressed trie using numbers - (**Indexes**)
- The indexes of the word is ...

0	1	2	3	4	5	6	7
m	**i**	**n**	**i**	**m**	**i**	**z**	**e**



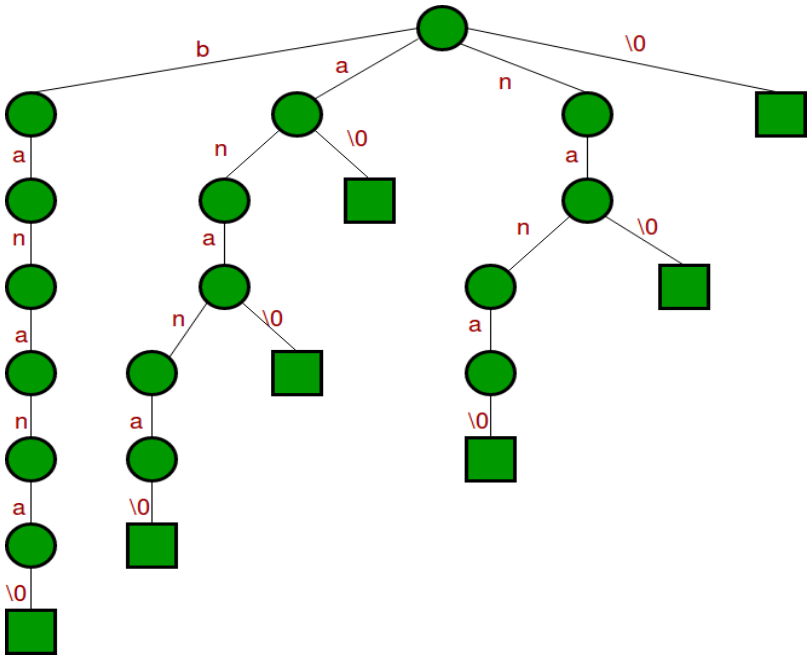
Suffix Trees – Introduction

- A simple data structure for string searching
- It's a compressed Trie Tree
- Allow many fast implementations of many important string operations

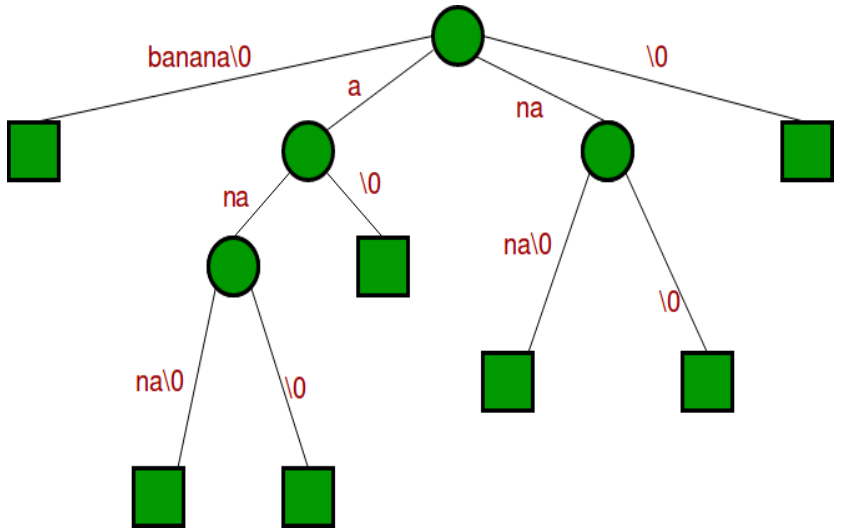
- Properties of a suffix trees:
  - ✓ A suffix tree for a text X of size n from an alphabet of size d.
  - ✓ Stores all the n(n-1) suffixes of X.
  - ✓ Supports arbitrary pattern matching and prefix matching queries

Suffix Trees – Introduction

- Join chains of single nodes,to get the following compressed trie, which is the Suffix tree for given text “banana\0”



Suffix TREE



Search for a substring in a Suffix Tree

Algorithm Steps

Step 1: Start from the **first character** of the pattern and the **root** of the suffix tree.

Step 2: For **each character** in the pattern:

If there is an **edge** from the current node labeled with this character, **follow the edge**.

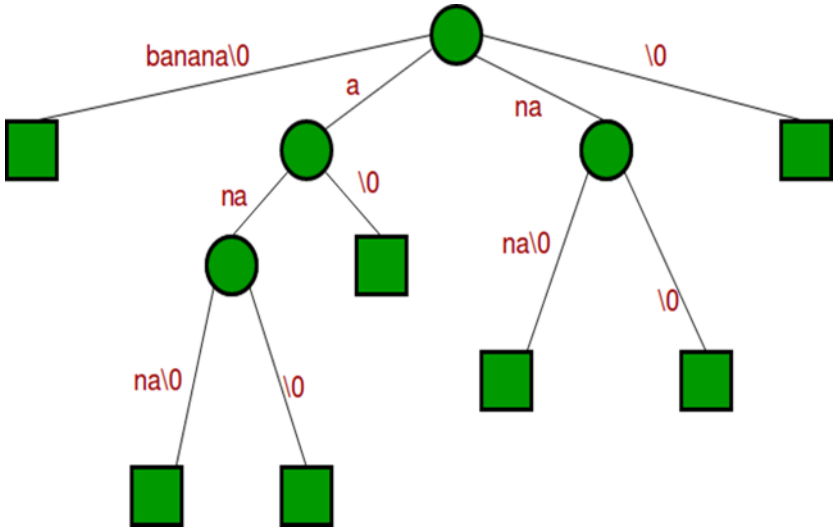
If there is **no such edge**, display

➤ *"Pattern doesn't exist in text"* and **stop**.

Step 3: If **all characters** of the pattern are processed successfully — i.e., there exists a **path from the root** corresponding to all characters in the pattern —

display

➤ *"Pattern found."*



1. Trie is also known as what?

a) digital tree
b) treap
c) binomial tree
d) 2-3 tree

1. Trie is also known as what?

a) digital tree
b) treap
c) binomial tree
d) 2-3 tree

2. What is NOT true about a Trie?

a) trie requires less storage space than hashing
b) trie allows listing all words with the same prefix
c) tries are collision free
d) trie also known as prefix tree

2. What is NOT true about a Trie?

a) trie requires less storage space than hashing
b) trie allows listing all words with the same prefix
c) tries are collision free
d) trie also known as prefix tree

3. What can be the maximum depth of a Trie given n strings and m is the max string length?

a) log n
b) log m
c) n
d) m

3. What can be the maximum depth of a Trie given n strings and m is the max string length?

a) log n
b) log m
c) n
d) m

4. Which of the following is true about a Trie?

a) root is letter a
b) path from root to leaf yields the string
c) children of nodes are randomly ordered
d) each node stores the associated keys

4. Which of the following is true about a Trie?

a) root is letter a
b) path from root to leaf yields the string
c) children of nodes are randomly ordered
d) each node stores the associated keys

THANK YOU

V R BADRI PRASAD

Department of Computer Science & Engineering

badriprasad@pes.edu

Data Structures and its Applications

V R BADRI PRASAD

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Implementation of TRIE Trees

V R BADRI PRASAD

Department of Computer Science & Engineering

TRIE Trees – Implementation

Structure of a node in a TRIE Tree :

- A node of a TRIE tree is represented as shown below.
- One field for each alphabet(A – Z), 26 columns.
- Each column is a pointer to another TRIE node or carries NULL and
- One field for end of word (key).

A	B	C	D	E	F	.....	W	X	Y	Z
F1	F2	F3	F4	F5	F6	.....	F23	F24	F25	F26
End of Word / (eok)										

Address of the next node (reference for us)
Field number – for user's reference no field is created , No memory is allocated
End of word / key field

```
struct trienode
{
  struct trienode* child[26];
  int endofword;
};
```

TRIE Trees – Implementation

```
struct trienode
{
   struct trienode* child[26];
   int endofword;
};
```

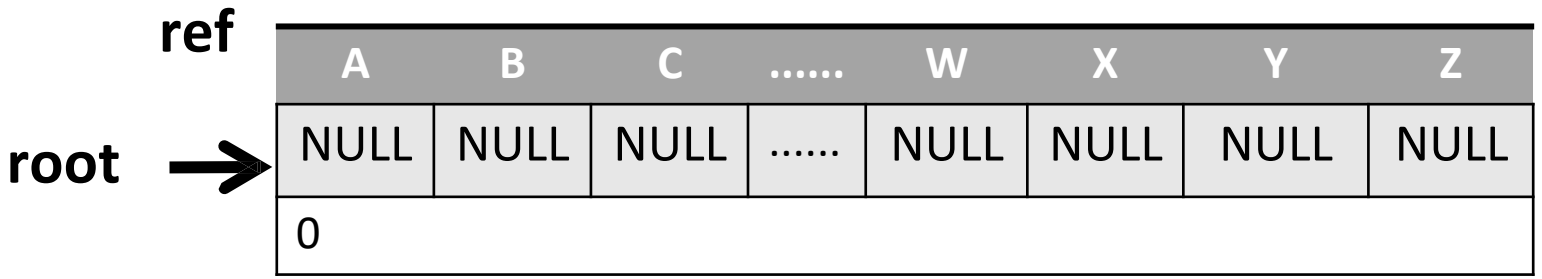
A	B	C	D	E	F	.....	W	X	Y	Z
F1	F2	F3	F4	F5	F6	.....	F23	F24	F25	F26
End of Word / (eok - $)										

Address of the next node (reference for us)
Field number
End of word / key field

TRIE Trees – Implementation

Creation of a node in a TRIE Tree using malloc() function.

```
struct trienode *getnode()
 {
  int i;
  struct trienode *temp;
   temp=(struct trienode*)(malloc(sizeof(struct trienode)));
   for(i=0;i<26;i++)
      temp->child[i]=NULL;        // Initialize all the fields to NULL.
   temp->endofword=0;          // Initialize endofword to 0.
  return temp;
 }
```



root=getnode();

TRIE Trees – Implementation

Insertion of a node in a TRIE Tree

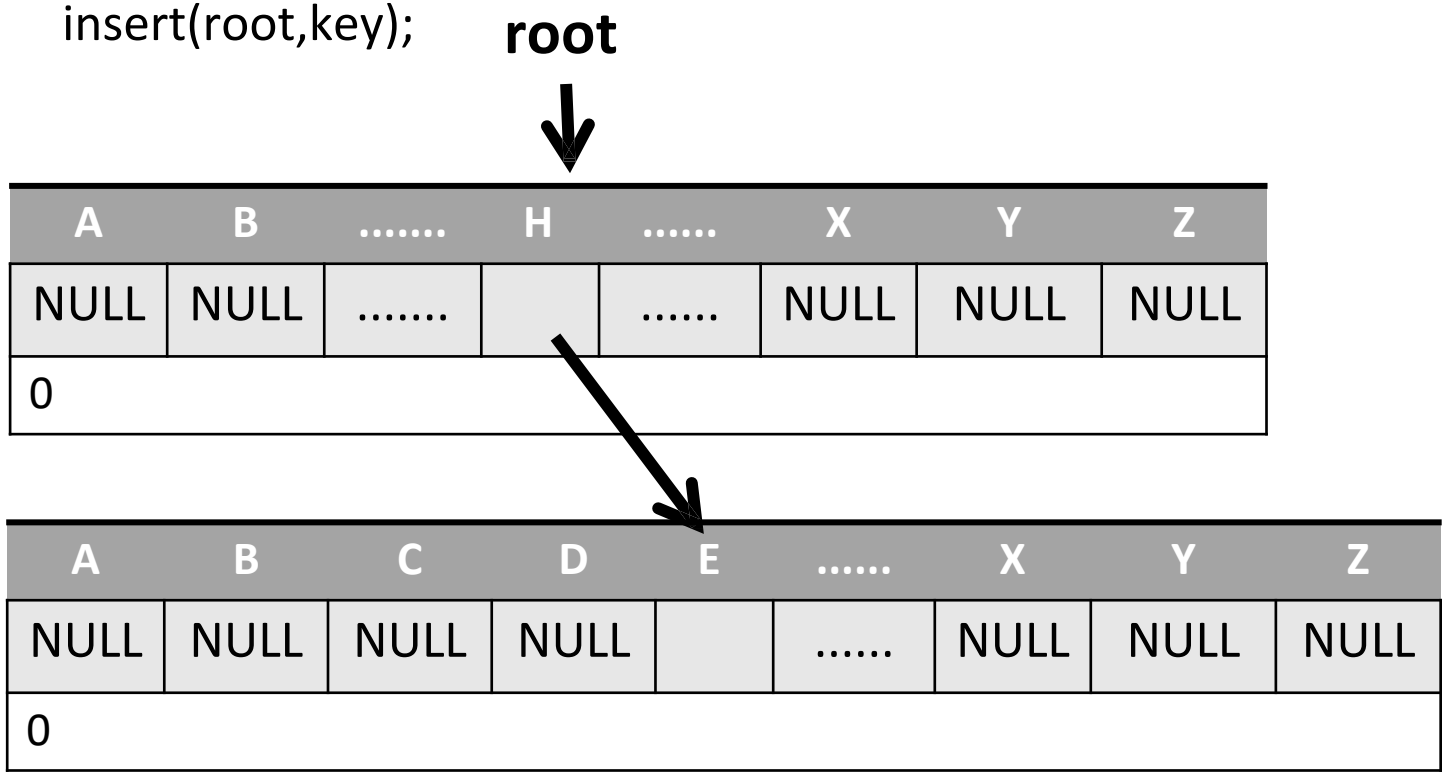
```
void insert(struct trienode *root, char *key)
 {
   struct trienode *curr; int i,index;

   curr=root;

   for(i=0;key[i]!='\0';i++)
   {
     index=key[i]-'a';
     if(curr->child[index]==NULL)
        curr->child[index]=getnode();
     curr=curr->child[index];
   }
  curr->endofword=1;
 }
```

TRIE Trees – Implementation

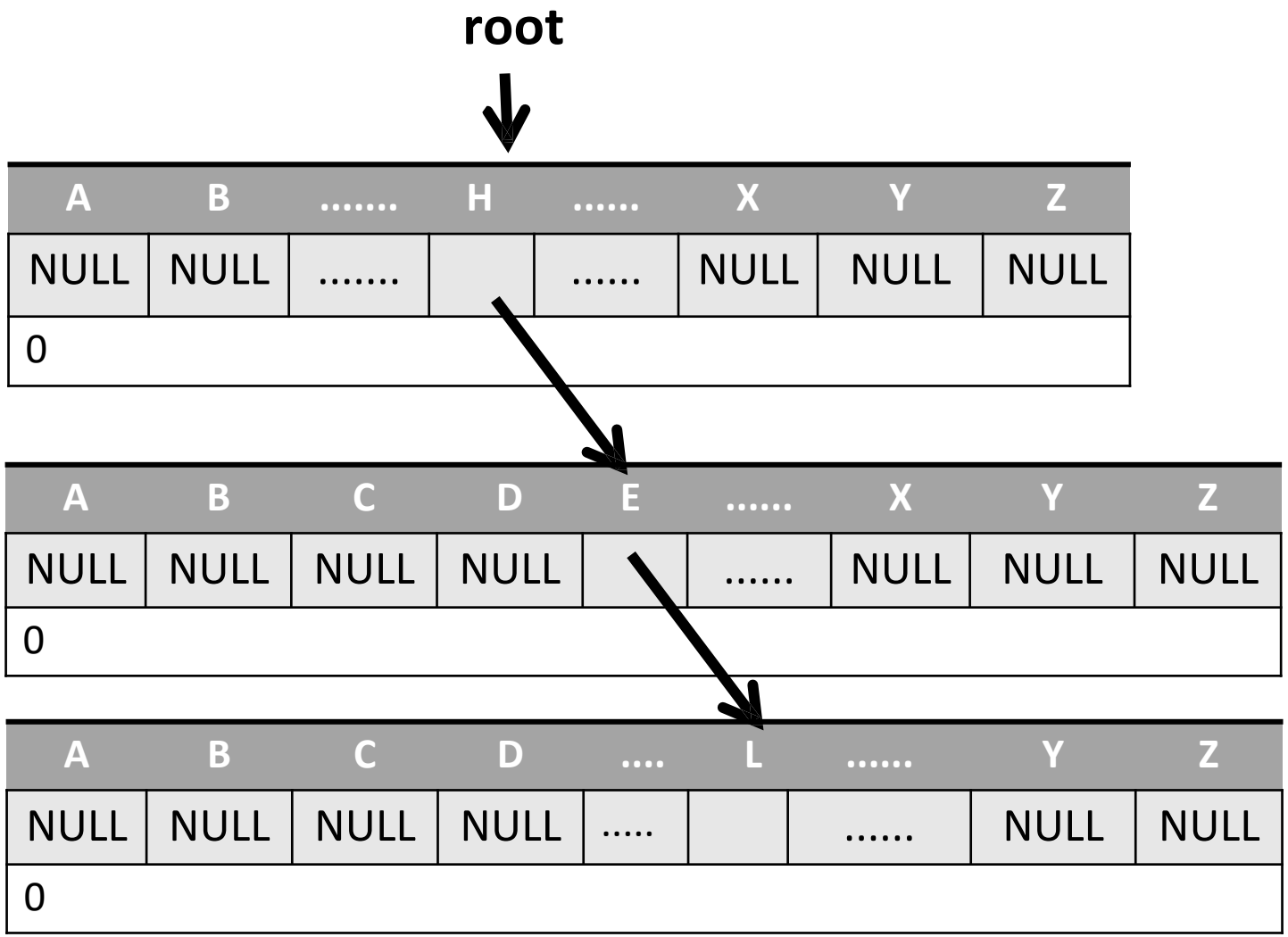
Insertion of a node in a TRIE Tree

On function call insert, the given string “HELLO” is inserted into the TRIE tree as shown below.

insert(root,key);



TRIE Trees – Implementation



1. In a Trie implemented for lowercase English letters, what does each node typically contain?

a) array of 26 pointers to children
b) array of 52 pointers to children
c) list all words
d) single pointer to parent

1. In a Trie implemented for lowercase English letters, what does each node typically contain?

a) array of 26 pointers to children
b) array of 52 pointers to children
c) list all words
d) single pointer to parent

2. What operation does the 'insert' method perform in a Trie?

a) adds a string character by character
b) removes a string
c) counts number of nodes
d) sorts all sorted strings

2. What operation does the 'insert' method perform in a Trie?

a) adds a string character by character
b) removes a string
c) counts number of nodes
d) sorts all sorted strings

3. How is prefix searching efficiently supported in a Trie?

a) by traversing nodes corresponding to prefix characters
b) using hash functions
c) sorting all entries beforehand
d) binary search on nodes

3. How is prefix searching efficiently supported in a Trie?

a) by traversing nodes corresponding to prefix characters
b) using hash functions
c) sorting all entries beforehand
d) binary search on nodes

4. During Trie insertion, what happens if the current character's child node does not exist?

a) create a new node for that character and link it
b) restart insertion from root
c) delete the entire root
d) ignore the character

4. During Trie insertion, what happens if the current character's child node does not exist?

a) create a new node for that character and link it
b) restart insertion from root
c) delete the entire root
d) ignore the character

5. After inserting all characters of a word into a Trie, what is the next step?

a) mark the last nodes as end of word
b) delete the last node
c) mark the root as end of word
d) None

5. After inserting all characters of a word into a Trie, what is the next step?

a) mark the last nodes as end of word
b) delete the last node
c) mark the root as end of word
d) None

THANK YOU

V R BADRI PRASAD

Department of Computer Science & Engineering

badriprasad@pes.edu

Data Structures and its Applications

V R BADRI PRASAD

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Implementation of TRIE Trees :

- Display Operation
- Deletion Operation
- Search Operation

V R BADRI PRASAD

Department of Computer Science & Engineering

TRIE TREES – Display function

```
void display(struct trienode *curr)
 {
   int i,j;
   for(i=0;i<255;i++)
   {
     if(curr->child[i]!=NULL)
     {
        word[length++]=i;
        if(curr->child[i]->endofword==1)//if end of word
        {
          printf("\n");
          for(j=0;j<length;j++)
          printf("%c",word[j]);
        }
       display(curr->child[i]);
        }
     }
   length--;
   return ;
}
```

TRIE Trees – Deletion Operation

```
void delete_trie(struct trienode *root, char *key)
 {
  int i,index,k;
  struct trienode *curr; struct stack x;
  curr=root;

  for(i=0;key[i]!='\0';i++)
  {
   index=key[i];
   if(curr->child[index]==NULL)
   {
      printf("The word not found..\n");
      return;
    }
   push(curr,index);
   curr=curr->child[index];
  }
  curr->endofword=0;
  push(curr,-1);
```

```
while(1)
 {
  x=pop();
  if(x.index!=-1)
    x.m->child[x.index]=NULL;
  if(x.m==root)//if root
    break;
  k=check(x.m);
  if((k>=1)||(x.m->endofword==1))
   break;
  else
    free(x.m);
 }
 return;
}
```

TRIE Trees – Search Operation


```
int search(struct trienode * root,char *key)
 {

 int i,index;
  struct trienode *curr; curr=root;

  for(i=0;key[i]!='\0';i++)
  {
    index=key[i];

    if(curr->child[index]==NULL) return 0;
     curr=curr->child[index];
   }
   if(curr->endofword==1) return 1;
  return 0;
}
```

1. In a Trie, when deleting a word that is a prefix of another word, what happens?

a) The entire Trie is deleted
b) Only the prefix nodes corresponding to the word are deleted
c) The word is removed, but shared nodes remain
d) Deletion fails if the word is a prefix

1. In a Trie, when deleting a word that is a prefix of another word, what happens?

a) The entire Trie is deleted
b) Only the prefix nodes corresponding to the word are deleted
c) The word is removed, but shared nodes remain
d) Deletion fails if the word is a prefix

2. During Trie deletion, when should a node be removed from the Trie?

a) When the node has children
b) When the node is the end of another word
c) When the node has no children and is not the end of a word
d) When the node has siblings

2. During Trie deletion, when should a node be removed from the Trie?

a) When the node has children
b) When the node is the end of another word
c) When the node has no children and is not the end of a word
d) When the node has siblings

3. When deleting a word from a Trie, which of the following must be checked to decide if a node can be deleted?

a) Whether the node has children
b) Whether the node marks the end of another word
c) Both a and b
d) Neither a nor b

3. When deleting a word from a Trie, which of the following must be checked to decide if a node can be deleted?

a) Whether the node has children
b) Whether the node marks the end of another word
c) Both a and b
d) Neither a nor b

4. During Trie deletion, what should be done when the node is not end of any other word and has no children?

a) Keep the node
b) Mark it as deleted but keep in Trie
c) Remove the node to free memory
d) Replace it with a new node

4. During Trie deletion, what should be done when the node is not end of any other word and has no children?

a) Keep the node
b) Mark it as deleted but keep in Trie
c) Remove the node to free memory
d) Replace it with a new node

THANK YOU

V R BADRI PRASAD

Department of Computer Science & Engineering

badriprasad@pes.edu

Data Structures and its Applications

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Word prediction using Prefix Trie and Suffix Trie

Department of Computer Science & Engineering

Prefix Trie vs Suffix Trie

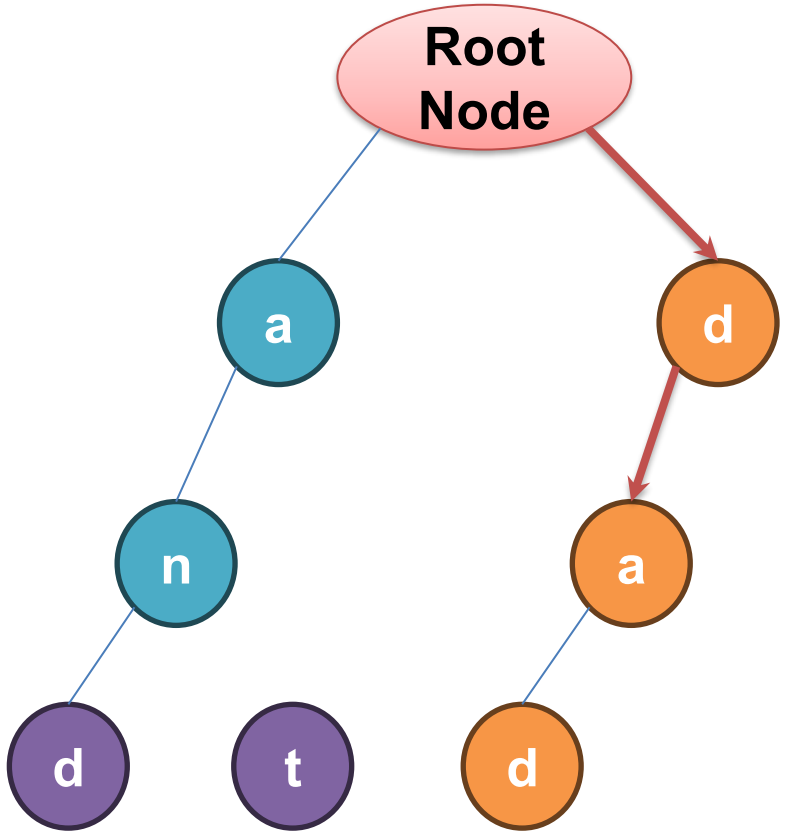
Feature	Prefix Trie	Suffix Trie
Definition	Stores all words or strings as-is, focusing on prefixes.	Stores all suffixes of a string for efficient substring search.
Purpose	Fast prefix lookup	Fast substring and pattern match
Construction	Insert complete words from start to end.	Insert all suffixes of a string.
End Marker	$ to mark end of word.	$ to mark end of each suffix.
Use Case	Autocomplete, Dictionary Search	Pattern Matching, Substring Search, Bioinformatics
Example (string “data”)	Insert *data* : root – d – a – t – a - $	Insert suffixes : “data”, “ata”,”ta”,”a”

Prefix Trie vs Suffix Trie

- Prefix tries and suffix tries are both tree-like structures for storing strings, but they differ in purpose and construction.
- A prefix trie stores complete words starting from their first character, sharing nodes for common prefixes, and is optimized for prefix searches like autocomplete or dictionary lookup.
- In contrast, a suffix trie stores all possible suffixes of a string, with each suffix inserted separately, making it larger but suitable for substring searches and pattern matching.

Word prediction using Prefix Trie

- Searches using a prefix, not a full word
- Traversal stops after matching all prefix characters
- Example: stored words – and, ant, dad
- User types ‘da’ → path: d → a
- Path exists ⇒ words starting with ‘da’ found
- Suggested words: dad
- Efficient for word prediction / auto-complete



Word prediction using Suffix Trie

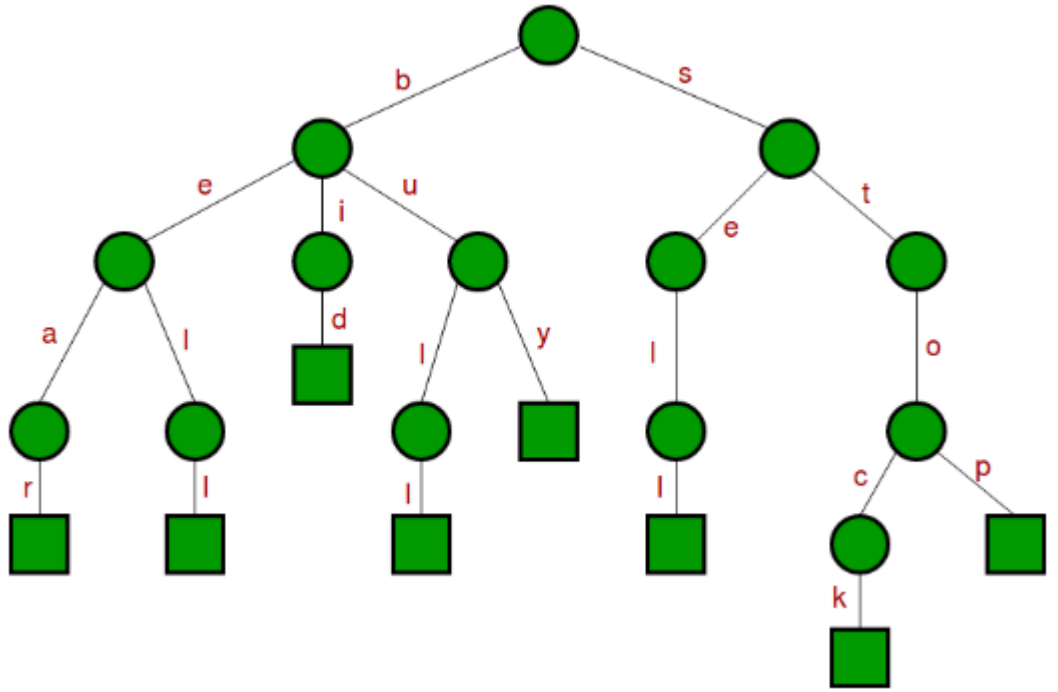
A Suffix Trie stores all suffixes of each word so that predictions can be made even from the middle or end of a word.

Example:

- From ‘bear’, suffixes like ‘ear’, ‘ar’, and ‘r’ are stored.
- If user types ‘ar’, the Suffix Trie can suggest ‘bear’.
- If user types ‘op’, it matches the suffix in ‘stop’ → suggestion ‘stop’.

Thus, Suffix Tries are useful for predictions based on endings or partial substrings, unlike Prefix Tries that predict only from word beginnings.

{bear, bell, bid, bull, buy, sell, stock, stop}



Data Structures and its Applications

Multiple-Choice-Questions (MCQ's)

1. In a Prefix Trie, prediction is based on:

a) Ending of word

b) Middle substring

c) Starting characters of the word

d) Random traversal

Data Structures and its Applications

Multiple-Choice-Questions (MCQ's)

1. In a Prefix Trie, prediction is based on:

a) Ending of word

b) Middle substring

c) Starting characters of the word

d) Random traversal

2. Which data structure is more suitable for predicting words when only the beginning characters are typed?

a) Hash table

b) Prefix trie

c) Suffix trie

d) Stack

2. Which data structure is more suitable for predicting words when only the beginning characters are typed?

a) Hash table

b) Prefix trie

c) Suffix trie

d) Stack

3. Given the word "stock", which of the following will be stored in a Suffix Trie?

a) s, st, sto

b) stock, tock, ock, ck,k

c) s, t, o, c, k

d) None of these

3. Given the word "stock", which of the following will be stored in a Suffix Trie?

a) s, st, sto

b) stock, tock, ock, ck,k

c) s, t, o, c, k

d) None of these

4. In a Suffix Trie, the word "bell" will generate which substring?

a) bel

b) ell

c) be

d) lbe

4. In a Suffix Trie, the word "bell" will generate which substring?

a) bel

b) ell

c) be

d) lbe

5. Which Trie structure takes more space due to storing more combinations?

a) Prefix trie

b) Suffix trie

c) Both take equal space

d) None

5. Which Trie structure takes more space due to storing more combinations?

a) Prefix trie

b) Suffix trie

c) Both take equal space

d) None

References

1. https://www.geeksforgeeks.org/dsa/trie-insert-and-search/
2. https://www.geeksforgeeks.org/dsa/pattern-searching-using-suffix-tree/

THANK YOU

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Kusuma K V

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing

Kusuma K V
Department of Computer Science & Engineering

Hashing

Time taken for search

- Linear DS: Array List, Linked List --- O(n)
- Non linear DS: Balanced Binary Search Tree --- O(log n)
- Can we search in O(1) time ??
  - Hashing

Hashing

Hashing is the process of transforming any given key into another value.

This is usually represented by a shorter, fixed-length value or key that makes it easier to find the original key.

The most popular use for hashing is the implementation of hash tables.

A hash table stores key and value pairs in a list that is accessible through its index.

Hashing

Direct Mapping

Eg: Employee Records

Hashing

Hash function: A function which maps key value into a hash table range

- Folding
- Truncation
- Modulo

Hashing

Collision: The phenomenon when two or more keys generate the same hash.

Collision resolution:

- Open addressing (Closed Hashing)
- Separate Chaining (Open Hashing)

Hashing

Collision resolution:

- Open addressing (Closed Hashing)
- Open addressing handles collisions by storing all data in the hash table itself and then seeking out availability in the next spot created by the algorithm.
    - Linear Probing, Quadratic Probing, Double Hashing

- Separate Chaining (Open Hashing)
- Separate chaining handles collision by making every hash table cell point to linked lists of records with identical hash function values.

Hashing

Collision resolution:

- Rehashing (Hash again)
  - Increase the table size when the load factor becomes more than a predefined value (Say 0.75) and hash all the keys present in the table again and store in the new table of increased size.

  Load factor=No. of filled records/Total capacity

1. What is the main purpose of hashing in data structures?

a) to sort data efficiently
b) to search and access data quickly
c) to compress data
d) to encrypt data for security

1. What is the main purpose of hashing in data structures?

a) to sort data efficiently
b) to search and access data quickly
c) to compress data
d) to encrypt data for security

2. In a hash table, what is the purpose of a hash function?:

a) to sort data
b) to compress data
c) to compute index for storing data
d) to delete data

2. In a hash table, what is the purpose of a hash function?:

a) to sort data
b) to compress data
c) to compute index for storing data
d) to delete data

3. Which method stores collided elements in a linked list at the hashed location?

a) linear probing
b) dynamic hashing
c) separate chaining
d) open addressing

3. Which method stores collided elements in a linked list at the hashed location?

a) linear probing
b) dynamic hashing
c) separate chaining
d) open addressing

4. The disadvantage of linear probing is:

a) increased clustering
b) increased memory usage
c) slow insertion
d) poor searching accuracy

4. The disadvantage of linear probing is:

a) increased clustering
b) increased memory usage
c) slow insertion
d) poor searching accuracy

5. Which collision resolution method minimizes primary clustering?

a) linear probing
b) quadratic probing
c) separate chaining
d) None

5. Which collision resolution method minimizes primary clustering?

a) linear probing
b) quadratic probing
c) separate chaining
d) None

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS

Kusuma K V

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Collision Resolution
Separate Chaining

Kusuma K V
Department of Computer Science & Engineering

Hashing: Separate Chaining

Separate Chaining (Open hashing) handles collision by making every hash table cell point to linked lists of records with identical hash function values.

Hashing: Separate Chaining

Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7.

Use key%tableSize as the hash function and separate chaining (open hashing) to resolve collision.

h(7) = 7 % 7 = 0

h(20) = 20 % 7 = 6

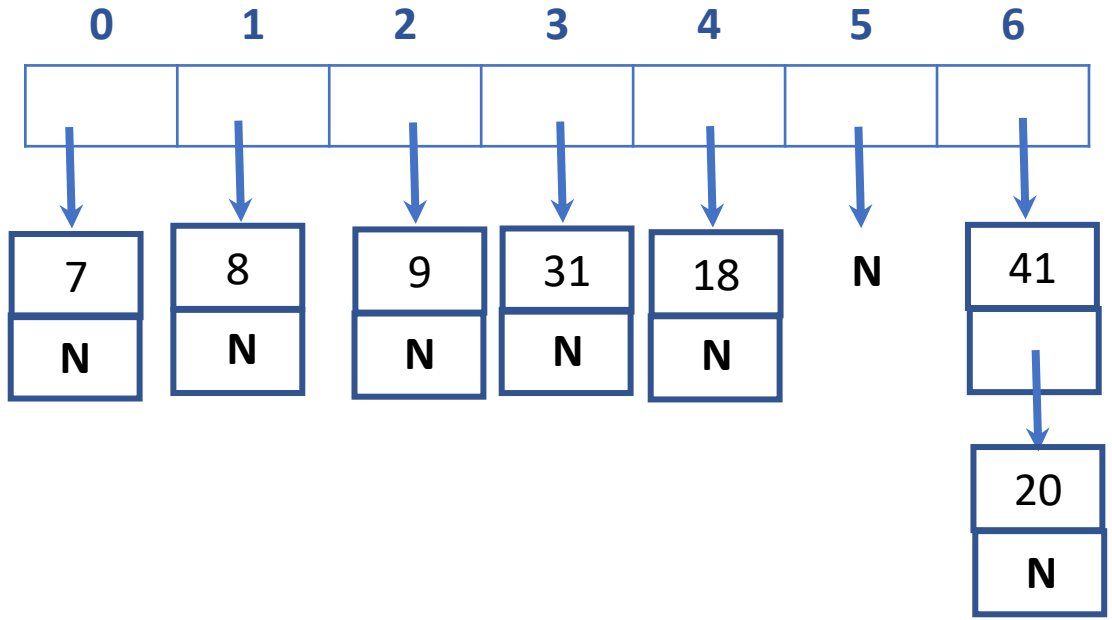
h(41) = 41 % 7 = 6 collision

h(31) = 31 % 7 = 3

h(18) = 18 % 7 = 4

h(8) = 8 % 7 = 1

h(9) = 9 % 7 = 2



Hashing: Separate Chaining


```
typedef struct element
{
        int rNo;
        char name[30];
        struct element *next;
}NODE;
```

Hashing: Separate Chaining

```
void initTable(NODE* ht[SIZE])
{
        for(int i=0;i<SIZE;i++)
                ht[i]=NULL;
}
```

Hashing: Separate Chaining

```
void insert(NODE* ht[SIZE],int rNo,char name[30])
{
        int index=rNo%SIZE;     //hash function

        NODE *newNode=malloc(sizeof(NODE));
        newNode->rNo=rNo;
        strcpy(newNode->name,name);
        newNode->next=ht[index];

        ht[index]=newNode;
}
```

Hashing: Separate Chaining

```
int delete(NODE* ht[SIZE],int rNo)
{
        int index=rNo%SIZE;  //hash function

        NODE *p=ht[index];
        NODE *q=NULL;

        while(p!=NULL && p->rNo!=rNo)
        {
                q=p;
                p=p->next;
        }
        if(p!=NULL)
        {
                if(q==NULL)
                        ht[index]=p->next;
                else
                        q->next=p->next;

                free(p);
                return 1;
        }
        return 0;
}
```

Hashing: Separate Chaining

```
int search(NODE* ht[SIZE],int rNo,char name[30]) {
        int index=rNo%SIZE;     //hash function
        NODE *p=ht[index];
        while(p!=NULL) {
                if(p->rNo==rNo) {
                        strcpy(name,p->name);
                        return 1;
                }
                p=p->next;
        }
        return 0;
}
```

1. What data structure is typically used in separate chaining to handle collisions in hash tables?

a) Array
b) Linked list
c) Stack
d) queue

1. What data structure is typically used in separate chaining to handle collisions in hash tables?

a) Array
b) Linked list
c) Stack
d) queue

2. In separate chaining, when a collision occurs at an index, what happens to the new key?

a) It replaces the existing key
b) It is inserted in the linked list at that index
c) It is places in the new empty index
d) It is discarded

2. In separate chaining, when a collision occurs at an index, what happens to the new key?

a) It replaces the existing key
b) It is inserted in the linked list at that index
c) It is places in the new empty index
d) It is discarded

3. Which of the following is an advantage of separate chaining over open addressing?

a) No extra memory needed
b) Handles large number of collisions without clustering
c) Search is always constant time
d) No linked list traversal required

3. Which of the following is an advantage of separate chaining over open addressing?

a) No extra memory needed
b) Handles large number of collisions without clustering
c) Search is always constant time
d) No linked list traversal required

4. If a hash table uses separate chaining and has many more elements than slots, the average search time becomes proportional to:

a) Number of slots
b) Load factor(number of elements/slots)
c) Constant time
d) Square number of elements

4. If a hash table uses separate chaining and has many more elements than slots, the average search time becomes proportional to:

a) Number of slots
b) Load factor(number of elements/slots)
c) Constant time
d) Square number of elements

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS

Kusuma K V

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Collision Resolution
Linear Probing

Kusuma K V
Department of Computer Science & Engineering

Hashing: Linear Probing

Linear Probing (open addressing, closed hashing) resolves collision by finding the next vacant spot in the hash table, in other words, it uses the below formula to resolve collision:

h(key) = (h(key) + i) % tableSize

where i = 1, 2, 3, …

Hashing: Linear Probing

Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7.

Use key%tableSize as the hash function & linear probing(closed hashing)to resolve collision.

h(7) = 7 % 7 = 0

h(20) = 20 % 7 = 6

h(41) = 41 % 7 = 6 collision,

0	1	2	3	4	5	6
7	41	8	31	18	9	20

Resolving collision: h(41)=((6+1)%7)=0, h(41)=((6+2)%7)=1

h(31) = 31 % 7 = 3

h(18) = 18 % 7 = 4

h(8) = 8 % 7 = 1 collision

Resolving collision: h(8)=((1+1)%7)=2

h(9) = 9 % 7 = 2 collision

Resolving collision: h(9)=((2+1)%7)=3, h(9)=((2+2)%7)=4, h(9)=((2+3)%7)=5

Hashing: Linear Probing

```
void initTable(NODE ht[],int *n)
{
        for(int i=0;i<SIZE;i++)
                ht[i].mark=-1;

        *n=0;
}
```

Hashing: Linear Probing

```
int insertRecord(NODE ht[],int rNo,char name[],int *n) {
        if(SIZE==*n)
                return 0;
        int index=rNo%SIZE;             //hash function
        while(ht[index].mark!=-1)
                index=(index+1)%SIZE;
        ht[index].rNo=rNo;
        strcpy(ht[index].name,name);
        ht[index].mark=1;
        (*n)++;
        return 1;
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Linear Probing

```
int deleteRecord(NODE ht[],int rNo,int *n){
    if(*n==0)
            return 0;

    int index=rNo%SIZE;        //hash function
    int i=0;
    while(ht[index].rNo!=rNo && i<*n)
    {
            index=(index+1)%SIZE;
            if(ht[index].mark==1)
                    i++;
    }
```

```
            if(ht[index].rNo==rNo                    &&
ht[index].mark==1)      {
                    ht[index].mark=-1;
                    (*n)--;
                    return 1;
            }
            return 0;
          }
```

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Linear Probing

```
int searchRecord(NODE ht[],int rNo,int n){
    if(n==0)
            return 0;

    int index=rNo%SIZE;        //hash function
    int i=0;
    while(ht[index].rNo!=rNo && i<n)
    {
            index=(index+1)%SIZE;
            if(ht[index].mark==1)
                    i++;
    }
```

```
            if(ht[index].rNo==rNo                    &&
ht[index].mark==1)      {
                    strcpy(name,ht[index].name);
                    return 1;
            }
            return 0;
          }
```

1. Which of the following statements about linear probing is true?

a) It eliminates clustering completely
b) It leads to primary clustering
c) It uses separate linked lists for collision resolution
d) It is slower than quadratic probing in search

1. Which of the following statements about linear probing is true?

a) It eliminates clustering completely
b) It leads to primary clustering
c) It uses separate linked lists for collision resolution
d) It is slower than quadratic probing in search

2. In linear probing, if a collision occurs at index h(k), the next slot checked is:

a) (h(k) + 2) % table_size
b) (h(k) + 1) % table_size
c) (h(k) * 2) % table_size
d) (h(k) - 1) % table_size

2. In linear probing, if a collision occurs at index h(k), the next slot checked is:

a) (h(k) + 2) % table_size
b) (h(k) + 1) % table_size
c) (h(k) * 2) % table_size
d) (h(k) - 1) % table_size

3. In linear probing collision resolution, if the hash table is full, what happens when a new key is inserted?

a) The key is discarded
b) The table size is increased automatically
c) The insertion fails due to overflow
d) The key is placed at the first index

3. In linear probing collision resolution, if the hash table is full, what happens when a new key is inserted?

a) The key is discarded
b) The table size is increased automatically
c) The insertion fails due to overflow
d) The key is placed at the first index

4. Consider a hash table of size 10 where keys 42, 23, 34, 52, 46, 33 are inserted using hash function h(k) = k mod 10 and linear probing. Which key will be placed at index 7?

a) 52
b) 33
c) 46
d) 23

4. Consider a hash table of size 10 where keys 42, 23, 34, 52, 46, 33 are inserted using hash function h(k) = k mod 10 and linear probing. Which key will be placed at index 7?

a) 52
b) 33
c) 46
d) 23

5. Given keys 661, 182, 24, and 103 inserted in order into a hash table of size 13 with h(k) = k mod 13 and linear probing, at which index will 103 be placed?

a) 0
b) 12
c) 1
d) 11

5. Given keys 661, 182, 24, and 103 inserted in order into a hash table of size 13 with h(k) = k mod 13 and linear probing, at which index will 103 be placed?

a) 0
b) 12
c) 1
d) 11

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS

Kusuma K V

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Collision Resolution
Quadratic Probing, Double Hashing

Kusuma K V
Department of Computer Science & Engineering

Hashing: Quadratic Probing

Quadratic Probing (Open addressing, closed hashing) resolves collision by using the below formula:

h(key) = (h(key) + i^2) % tableSize

where i = 1, 2, 3, …

Hashing: Quadratic Probing

Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7. Use key%tableSize as the hash function & quadratic probing to resolve collision.

h(7) = 7 % 7 = 0

h(20) = 20 % 7 = 6

h(41) = 41 % 7 = 6 collision

0	1	2	3	4	5	6
7	8	9	41	31	18	20

Resolving collision: h(41)=((6+1^2)%7)=0, h(41)=((6+2^2)%7)=3

h(31) = 31 % 7 = 3 collision

Resolving collision: h(31)=((3+1^2)%7)=4

h(18) = 18 % 7 = 4 collision

Resolving collision: h(18)=((4+1^2)%7)=5

h(8) = 8 % 7 = 1

h(9) = 9 % 7 = 2

Hashing: Double Hashing

Double Hashing (Open addressing, closed hashing) resolves collision by using a second hash function whenever there results a collision. The below formula is used:

hash(key) = (hash1(key) + i * hash2(key)) % tableSize

Here hash1() and hash2() are hash functions and tableSize is the size of hash table, i = 1, 2, …

Hashing: Double Hashing

Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7.

Use **key%tableSize** as the **first hash function** & double hashing (closed hashing) to resolve collision. Use **key%5** as the **second hash function**.

0	1	2	3	4	5	6
7	41		31	18		20

h(7) = 7 % 7 = 0

h(20) = 20 % 7 = 6

h(41) = 41 % 7 = 6 collision

Resolving collision: 2nd hash function: h(41) = 41 % 5 = 1

Therefore, h(41) = (6 + 1*1) % 7 = 0, h(41) = (6 + 2*1) % 7 = 1

h(31) = 31 % 7 = 3

h(18) = 18 % 7 = 4

Hashing: Double Hashing

Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7.

Use **key%tableSize** as the **first hash function** & double hashing (closed hashing)to resolve collision. Use **key%5** as the **second hash function**.

contd..,

0	1	2	3	4	5	6
7	41	8	31	18	9	20

h(8) = 8 % 7 = 1 collision

Resolving collision: 2nd hash function: h(8) = 8 % 5 = 3

Therefore, h(8) = (1 + 1*3) % 7 = 4, h(8) = (1 + 2*3) % 7 = 0, h(8) = (1 + 3*3) % 7 = 3

h(8) = (1 + 4*3) % 7 = 6, h(8) = (1 + 5*3) % 7 = 2

h(9) = 9 % 7 = 2 collision

Resolving collision: 2nd hash function: h(9) = 9 % 5 = 4

Therefore, h(9) = (2 + 1*4) % 7 = 6, h(9) = (2 + 2*4) % 7 = 3, h(9) = (2 + 3*4) % 7 = 0

h(9) = (2 + 4*4) % 7 = 4, h(9) = (2 + 5*4) % 7 = 1, h(9) = (2 + 6*4) % 7 = 5

1. Which of the following best describes quadratic probing in hashing?

a) It checks the next slot sequentially until an empty one is found
b) It uses a quadratic function to calculate the probe steps
c) It stores collided keys in a linked list at hash index
d) It uses two hash functions to calculate the probe sequence

1. Which of the following best describes quadratic probing in hashing?

a) It checks the next slot sequentially until an empty one is found
b) It uses a quadratic function to calculate the probe steps
c) It stores collided keys in a linked list at hash index
d) It uses two hash functions to calculate the probe sequence

2. Quadratic probing primarily helps

a) To reduce clustering
b) Increase clustering
c) To increase hash table size
d) To achieve external chaining

2. Quadratic probing primarily helps

a) To reduce clustering
b) Increase clustering
c) To increase hash table size
d) To achieve external chaining

3. Which Which collision resolution technique is considered the most effective in minimizing both primary and secondary clustering?

a) linear probing
b) quadratic probing
c) double hashing
d) separate chaining

3. Which Which collision resolution technique is considered the most effective in minimizing both primary and secondary clustering?

a) linear probing
b) quadratic probing
c) double hashing
d) separate chaining

4. If a hash table size m is prime, quadratic probing guarantees which property?

a) all slots can be probed before failure
b) primary clustering occurs
c) second clustering is eliminated
d) collisions never occur

4. If a hash table size m is prime, quadratic probing guarantees which property?

a) all slots can be probed before failure
b) primary clustering occurs
c) second clustering is eliminated
d) collisions never occur

5. Which statement about quadratic probing is true?

a) it always eliminates all collision problems
b) it may still suffer from secondary clustering
c) it uses two different hash functions
d) it stores collided items in linked lists

5. Which statement about quadratic probing is true?

a) it always eliminates all collision problems
b) it may still suffer from secondary clustering
c) it uses two different hash functions
d) it stores collided items in linked lists

THANK YOU

Kusuma K V

Department of Computer Science & Engineering

kusumakv@pes.edu

DATA STRUCTURES AND ITS APPLICATIONS

Department of Computer Science & Engineering
TA – Hannah Alex

PES UNIVERSITY

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Collision Resolution Using Rehashing

Department of Computer Science & Engineering

Hashing: Rehashing

Rehashing is the process of increasing the size of a hashmap and redistributing the elements to new buckets based on their new hash values. It is done to improve the performance of the hashmap and to prevent collisions caused by a high load factor (i.e., the ratio of the number of elements to the number of buckets).

Hashing: Rehashing

Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7.

Use key%tableSize as the hash function & linear probing(closed hashing)to resolve collision.

Perform rehashing (i.e., double the table size to the next prime number) when the load factor exceeds 0.7, and continue inserting the remaining keys into the new table.

h(7) = 7 % 7 = 0 (load factor<0.7)

h(20) = 20 % 7 = 6 (load factor<0.7)

h(41) = 41 % 7 = 6 collision,

0	1	2	3	4	5	6
7	41	-	31	18	-	20

Resolving collision: h(41)=((6+1)%7)=0, h(41)=((6+2)%7)=1 (load factor<0.7)

h(31) = 31 % 7 = 3 (load factor<0.7)

h(18) = 18 % 7 = 4 rehashing required because

Used places = 5 size = 7

Load factor = 5 / 7 = 0.71 > 0.7

Hashing: Rehashing

For rehashing,

Old table size = 7 Double = 14

Next prime number >=14 i.e 17

New table size = 17

Reinsert keys in original order

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
-	18	-	20	-	-	-	7	41	8	9	-	-	-	31	-	-

h(7)=7%17=7

h(20)=20%17=3

h(41)=41%17=7

Resolving collision: h(41) = (7+1)%17=8

h(31)=31%17=14

h(18)=18%17=1

h(8)=8%17=8

Resolving collision: h(8) = (8+1)%17=9

h(9)=9%17=9

Resolving collision: (9+1)%17=10

1. What is rehashing in a hash table?

A) Chaining elements together
B) Hashing elements for the first time
C) Increasing the size of the hash table and reinserting elements
D) Using the same hash function repeatedly

PES UNIVERSITY

1. What is rehashing in a hash table?

A) Chaining elements together

B) Hashing elements for the first time

C) Increasing the size of the hash table and reinserting elements

D) Using the same hash function repeatedly

2. When should rehashing be triggered in a hash table?

A) When load factor exceeds a threshold
B) When collisions never occur
C) When the table is empty
D) When all keys are distinct

2. When should rehashing be triggered in a hash table?

A) When load factor exceeds a threshold
B) When collisions never occur
C) When the table is empty
D) When all keys are distinct

3. Which of the following is a main purpose of rehashing?

A) Increase collisions

B) Reduce number of collisions after resizing

C) Decrease the size of the table

D) Slow down insertion

PES UNIVERSITY

3. Which of the following is a main purpose of rehashing?

A) Increase collisions

B) Reduce number of collisions after resizing

C) Decrease the size of the table

D) Slow down insertion

4.What typically happens during rehashing?

A) The hash function remains the same, elements are reinserted in a larger table

B) The hash function changes to a random function

C) Elements are sorted before reinserting

D) Elements are removed from the table permanently

4.What typically happens during rehashing?

A) The hash function remains the same, elements are reinserted in a larger table

B) The hash function changes to a random function

C) Elements are sorted before reinserting

D) Elements are removed from the table permanently

5. What is a possible downside of rehashing?

A) It requires additional memory and time to reinsert all elements
B) It eliminates collisions completely
C) It decreases the size of the table irreversibly
D) It prevents the use of open addressing methods

5. What is a possible downside of rehashing?

A) It requires additional memory and time to reinsert all elements

B) It eliminates collisions completely

C) It decreases the size of the table irreversibly

D) It prevents the use of open addressing methods

THANK YOU

Department of Computer Science & Engineering